

Détection et classification des balises cardinales

Image and Machine Learning Project (1)

FISE2 – P5 : Image & IA – Semestre 8 – 2025/2026

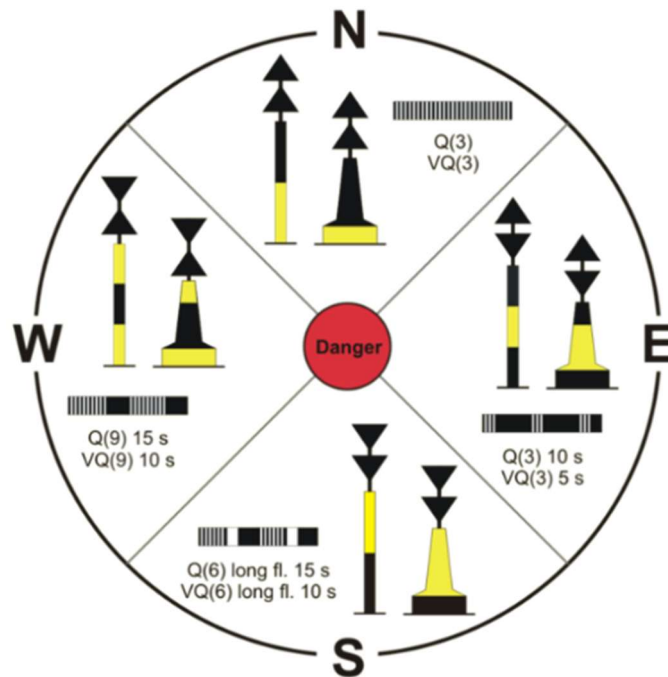


Table des matières

Description du projet.....	- 2 -
Contenu des notebooks.....	- 2 -
Approche 1 : Analyse d'image classique.....	- 3 -
Approche 2 : Machine Learning orienté analyse d'image	- 7 -
Approche 3 : Deep Learning orienté analyse d'image	- 11 -
Comparaison des approches	- 15 -
Scores	- 15 -
Interprétation	- 15 -
Coût computationnel.....	- 16 -
Application des approches pour une image test.....	- 16 -
Conclusion générale.....	- 19 -

Description du projet

L'objectif de ce projet est de développer des algorithmes pour détecter une balise cardinale et reconnaître sa direction cardinale en utilisant trois approches : l'analyse d'image classique (caractéristiques conçues manuellement, quelques techniques d'apprentissage automatique), l'analyse d'image orientée apprentissage automatique (caractéristiques conçues manuellement, algorithmes ML traditionnels) et l'analyse d'image orientée apprentissage profond (CNN, apprentissage par transfert).

Les deux tâches de reconnaissance à effectuer sont :

1. Détection de la balise cardinale dans l'image, consistant à identifier les coordonnées de la boîte englobante du marqueur (x, y, largeur, hauteur)
2. Classification de la direction cardinale, consistant à déterminer l'orientation correspondante (Ouest, Nord, Sud et Est) en interprétant la couleur ou les cônes.

Dans ce présent rapport, nous allons vous introduire nos 3 approches sur chacune des tâches de reconnaissance (détection et classification). A la fin de chaque approche, nous vous présenterons les résultats. Enfin, nous comparerons les différentes méthodes sous le plan des résultats et aussi du coût computationnel.

Nous affirmons que nous avons eu recours à l'intelligence artificielle générative mais qu'elle ne fait pas partie intégrante des programmes.

Contenu des notebooks

Les trois notebooks sont structurés de la même manière. On traite d'abord la détection des balises cardinales puis la classification des directions avec les balises recadrées par les boîtes englobantes vérité terrain. Après chacune de ces tâches, nous présentons nos résultats avec des métriques spécifiques à la tâche qui sont :

- Pour la détection :

$$IoU = \frac{\text{Intersection des aires des bounding boxes}}{\text{Union des aires des bounding boxes}}$$

$$mAP @ 0.50 = \frac{1}{N} \sum_{i=1}^N 1_{IoU_i > 0.5}$$

- Pour la classification :

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} \\ \text{Recall} &= \frac{TP}{TP + FN} \end{aligned}$$

$$\begin{aligned} \text{F1-Score} &= \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} \\ \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \end{aligned}$$

Approche 1 : Analyse d'image classique

Tâche de reconnaissance 1 : Détection de balise cardinale

Pour la détection de balise cardinale avec l'approche d'analyse d'image classique, on a appliqué la segmentation couleur vue le semestre dernier dans le module « Couleurs » et des opérations morphologiques simples revues dans le module « Advanced Image Analysis » du bloc « Image Science (1) ».

Démarche

Notre démarche est une suite logique d'opérations de traitement d'image classique :

1. Changement d'espace colorimétrique :

Pour commencer la démarche de détection, on choisit de créer des masques binaires. On commence alors par changer d'espace colorimétrique de RGB à HSV pour mieux extraire les teintes jaunes et sombres. En effet, pour la teinte jaune par exemple, il est plus simple de garder l'information de la teinte en sachant qu'elle peut changer de luminosité (jaune foncé, jaune clair).

2. Seuillage :

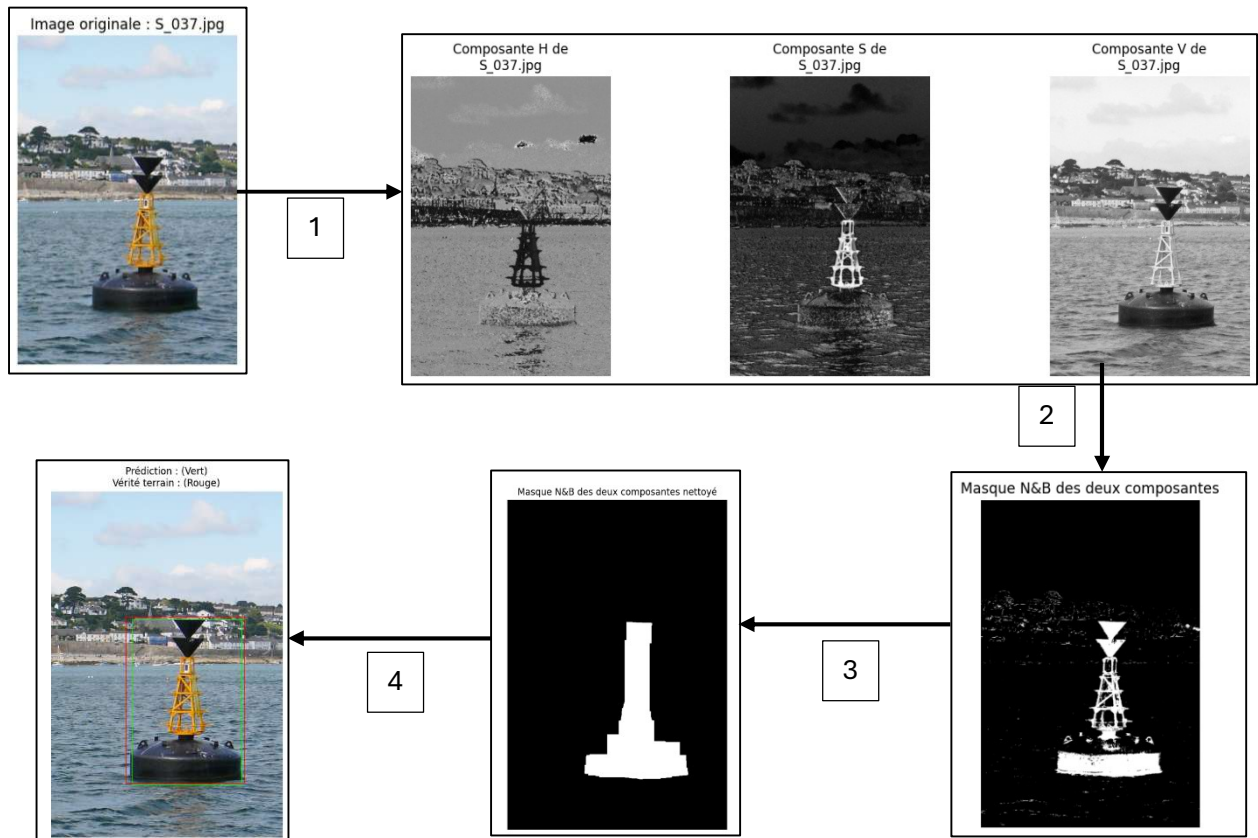
On crée deux masques binaires selon les composantes HSV. Un correspondant aux parties jaunes de l'image et un autre aux parties sombres de l'image. Concernant les valeurs, on a choisi les seuils $HSV_{jaune} = [15-35, 100-255, 100-255]$ et $HSV_{noir} = [0-180, 0-255, 0-60]$. H est la teinte et prend les valeurs entre 0 et 180 (correspond à la couleur), S est la saturation et prend les valeurs entre 0 et 255 (correspond à quel point la couleur est vive/fade) et V est la luminosité et prend les valeurs entre 0 et 255 (correspond à quel point la couleur est claire/sombre). On assemble ensuite les masques binaires avec l'opérateur « OR » pour avoir un masque binaire global afin de trouver la position approximative de la balise.

3. Nettoyage morphologique :

Sur certaines images, on peut apercevoir sur les masques des résidus dus au fond de l'image comportant la ville, de la verdure... On choisit alors de nettoyer le masque binaire par des opérations morphologiques. On applique d'abord une ouverture pour supprimer les points isolés issus du seuillage puis on applique une fermeture pour boucher les trous et relier les composantes séparées du masque binaire. Nous avons utilisé des éléments structurants rectangulaires de taille proportionnelle à celle de l'image. Nous prenons un élément structurant plus petit pour l'ouverture (environ 100 fois plus petit que l'image) que pour la fermeture (environ 10 fois plus petit que l'image) car on ne veut pas supprimer les détails fins pour l'image mais surtout les résidus.

4. Extractions des contours :

On cherche les composantes connexes qu'il reste et on choisit celle dont l'aire est la plus grande. Une des conditions à respecter est aussi que son ratio hauteur/largeur doit être entre 1.5 et 8 (car la balise est plus haute que large). Si aucune n'obéit à cette condition, on ignore la condition. A partir de cette composante, on peut facilement avoir la boîte englobante en effectuant une opération à base de min, max sur les coordonnées.



Résultats

On obtient pour les 167 images :

- Mean IoU: 0.5991
- mAP @ 0.5: 0.6886

Ce score est très satisfaisant car il prend en compte toutes les fois où les bounding boxes prédites sont en dehors des vraies (IoU = 0) et arrive quand même à atteindre presque 60% d'IoU moyenne.

Conclusion

Le principal avantage de cette méthode est son très faible coût computationnel. Contrairement aux réseaux de neurones, cet algorithme s'exécute extrêmement rapidement sur un CPU, ce qui est idéal pour un système embarqué en temps réel. De plus, chaque étape est transparente donc il est très facile de comprendre l'origine d'une erreur de détection et d'ajuster un paramètre manuellement.

Par contre, l'inconvénient majeur de cette technique est sa forte sensibilité aux variations environnementales. L'utilisation des seuils fixes qu'on a défini manuellement pour filtrer les couleurs rend l'algorithme vulnérable aux changements de luminosité ou aux forts reflets sur l'eau par exemple. Une image capturée sous un ciel très nuageux ou au crépuscule peut drastiquement altérer les teintes perçues et faire échouer la segmentation.

Tâche de reconnaissance 2 : Classification de direction cardinale

Pour la classification de direction cardinale avec l'approche d'analyse d'image classique, on a aussi appliqué la segmentation couleur vue le semestre dernier dans le module « Couleurs » et des opérations morphologiques simples revues dans le module « Advanced Image Analysis » du bloc « Image Science (1) ».

Démarche

Notre démarche est une suite logique d'opérations de traitement d'image classique :

1. Changement d'espace colorimétrique et seuillage :

On reprend le même procédé qu'à la détection avec les mêmes détails : On change d'espace colorimétrique de RGB à HSV puis on crée deux masques binaires correspondant à la composante jaune et à la composante sombre. Cependant, pour la classification, on ne combine pas les masques contrairement à la détection.

2. Nettoyage des masques :

Comme on part de la vérité terrain, on sait que l'image contient le haut du cône et le bas de la balise. On enlève une partie du haut (30%) pour négliger le cône car sa couleur biaise les résultats entre Est et Sud qui ont tous les deux un enchaînement $N > J > N$ où N est Noir et J est Jaune. On enlève aussi une partie du bas (10%) car elle peut comporter des éléments parasites comme de la mousse sombre sur la balise. On effectue ensuite une ouverture pour enlever les bruits et une fermeture pour boucher les trous comme détaillé à la détection.

3. Dilatation du masque :

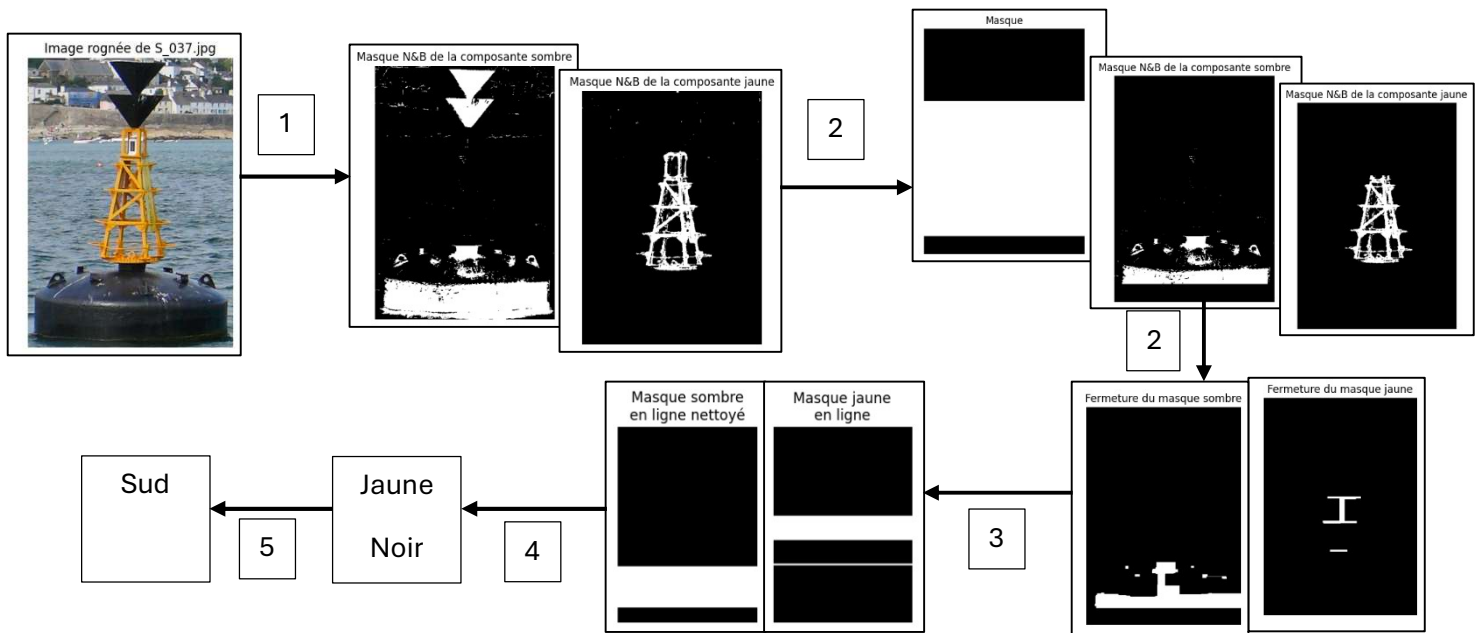
On effectue une dilatation des masques par un élément structurant ligne horizontale sur toute la largeur du masque. C'est une opération importante dans cette approche car on néglige la valeur de l'aire des différentes composantes connexes et de leur position horizontale en ne gardant que l'information de la hauteur des bandes. On enlève les bandes noires qui ont la même valeur que celles de jaune (car elles sont minoritaires habituellement).

4. Lecture de la séquence :

De haut en bas, on lit la séquence des masques noir et jaune en passant par les bandes blanches des masques (il est plus facile de voir par les images que de comprendre par les explications).

5. Décision :

La lecture de la séquence permet de connaître le cardinal correspondant (Nord : $N > J$, Sud : $J > N$, Est : $N > J > N$, Ouest : $J > N > J$).

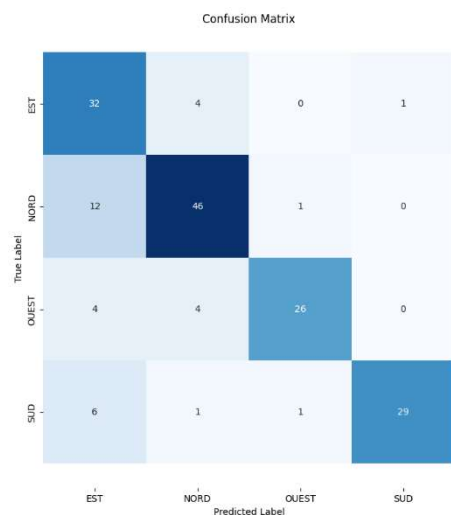


Résultats

On obtient pour les 167 images une accuracy globale de 79.64%.

On obtient la matrice de confusion ci-contre :

Les résultats sont très satisfaisants et surtout grâce à l'opération de dilatation de l'image avec un élément structurant ligne qui permet de prendre en compte seulement la hauteur des pixels et pas seulement de leur aire : l'accuracy globale a fait un bond de 60% à presque 80% avec cette opération.



Conclusion

Le grand avantage de cette méthode de classification est sa transparence totale. Cela rend la prise de décision instantanée en termes de temps de calcul, extrêmement facile à debug en cas d'erreur, et capable d'atteindre une excellente précision si la balise est bien isolée.

L'inconvénient majeur de cette classification est sa dépendance absolue à la qualité de la phase de détection qui la précède. Si le recadrage est imparfait et inclut un large bout de mer sombre en bas ou un ciel très lumineux en haut, les ratios de couleurs calculés seront faussés, ce qui entraînera une mauvaise prédiction.

On aurait aussi pu tenter une autre approche pour la classification en ne prenant que la partie haute de l'image segmentée (cône) et en déduire le cardinal correspondant en utilisant la transformée de Hough pour la reconnaissance de lignes.

Approche 2 : Machine Learning orienté analyse d'image

Tâche de reconnaissance 1 : Détection de balise cardinale

Pour la détection de balise cardinale avec l'approche de ML orienté analyse d'image, on a utilisé l'extraction de caractéristiques de HOG et la classification par SVM vue dans le TD10 du module « Advanced Image Analysis » du bloc « Image Science (1) ».

Démarche

Notre démarche suit la même approche que celui du TD c'est-à-dire :

1. Création de deux bases d'images :

Notre but est d'entraîner un classifieur SVM. On doit créer deux bases d'images d'entraînement dont une contenant les balises cardinales (classe positive) et une contenant des éléments du fond ne comportant pas la balise (classe négative) comme la mer ou le ciel par exemple. Pour la classe positive (balises cardinales), on choisit les morceaux de l'image grâce à leur boîte englobante qu'on a en guise de vérité terrain et on les redimensionne en taille 64x128 en pré-traitement pour l'extracteur de caractéristiques. Pour la classe négative, on choisit la fenêtre se situant en haut à gauche de l'image de taille 64x128 et dans ce cas, nous n'avons pas besoin de le redimensionner.

2. Extraction de caractéristiques :

A chaque image des deux bases, on lui attribue un vecteur de caractéristiques avec la méthode HOG (Histogram of Oriented Gradients) pour obtenir une information de sa structure spatiale et une classe (1 pour les balises et 0 pour le fond) pour la vérité terrain. On ajoute alors, image par image, à la liste 'X' le vecteur de caractéristiques d'une image et on ajoute à la liste 'y' sa classe c'est-à-dire '1' (dans le cas d'une balise) ou '0' (dans le cas du fond).

3. Entraînement du classifieur :

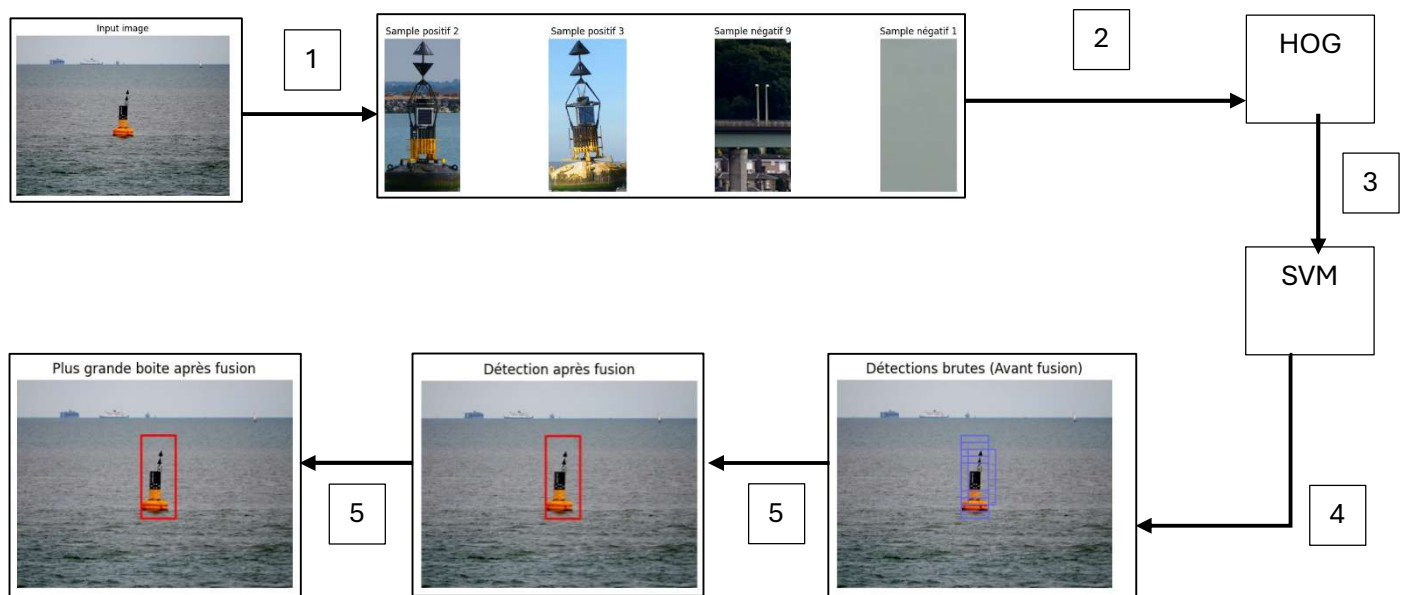
On entraîne le classifieur SVM avec la fonction LinearSVC (Linear Support Vector Classification) en lui donnant en argument 'X' et 'y' afin qu'il puisse séparer les vecteurs de caractéristiques des deux classes différentes.

4. Méthode de fenêtre glissante :

On parcourt l'image entière avec une image de taille fixe (64,128) pour que le classifieur puisse déterminer si la partie de l'image soumise à la fenêtre est une balise (classe 1) ou un fond (classe 0).

5. Post-traitement des boîtes :

Avec la méthode de la fenêtre glissante, on obtient les coordonnées des boîtes englobantes susceptibles de contenir le fond. On regroupe les ensembles de boîtes qui se chevauchent et on choisit celle qui a la plus grande aire en guise de prédiction (s'il y a plusieurs boîtes).



Résultats

On obtient pour les 84 images test :

- Mean IoU: 0.5393
- mAP @ 0.5: 0.7024

Malgré le temps d'exécution pour les images de test (5min), le résultat est satisfaisant avec une moyenne d'IoU de 53%.

Conclusion

Le principal atout de cette méthode est sa robustesse face aux variations d'illumination. En se concentrant sur les caractéristiques (HOG) plutôt que sur la couleur des pixels, l'algorithme est capable de détecter une balise même si elle est à contre-jour, partiellement dans l'ombre, ou que ses couleurs sont altérées par le reflet de l'eau.

Cependant, l'inconvénient majeur de cette technique, qui a été particulièrement flagrant lors de nos tests, est son coût computationnel. Le principe de la fenêtre glissante oblige le processeur à extraire les caractéristiques et à interroger le modèle SVM des milliers de fois pour analyser une seule image.

On aurait pu changer la façon de choisir les images des classes négatives car il est possible que certaines balises se situent en haut à gauche de l'image et qu'elles soient comptées comme négatives.

Tâche de reconnaissance 2 : Classification de direction cardinale

Pour la classification de direction cardinale avec l'approche de ML orienté analyse d'image, on a appliqué des opérations morphologiques simples et la méthode des K-Means vue dans les modules « Image Description » et « Advanced Image Analysis » du bloc « Image Science (1) ».

Démarche

Notre démarche est composée des étapes suivantes :

1. Seuillage couleur :

On reprend le même raisonnement que l'approche 1 mais nous restons dans l'espace couleur RGB. On décompose alors l'image selon ses 3 canaux RGB puis on crée un masque binaire jaune et un masque binaire noir avec des valeurs bien précises des composantes. Les valeurs du masque jaune sont $RGB_{\text{jaune}} = [0.4-1, 0.3-1, 0-0.3]$ car nous voulons une valeur importante pour la composante jaune (issu de rouge R et vert G) et les valeurs du masque noir sont $RGB_{\text{noir}} = [0-0.3, 0-0.5, 0-0.3]$ car les composantes de chaque canal doivent être faible.

2. Nettoyage des masques :

Comme pour la première approche, on part des boîtes englobantes de vérité terrain pour se focaliser seulement sur l'opération de classification. On enlève alors une partie du haut (30%) pour négliger les cônes et une partie du bas (10%) car elle peut comporter des éléments parasites (comme pour la première approche). On effectue une ouverture pour enlever les bruits mais pas de fermeture contrairement à la première approche.

3. Clustering :

On applique la méthode de K-Means sur chacun des masques avec deux clusters initialisés en haut au milieu et en bas au milieu afin de trouver les deux endroits où les pixels sont le plus concentré.

4. Post-traitement des clusters :

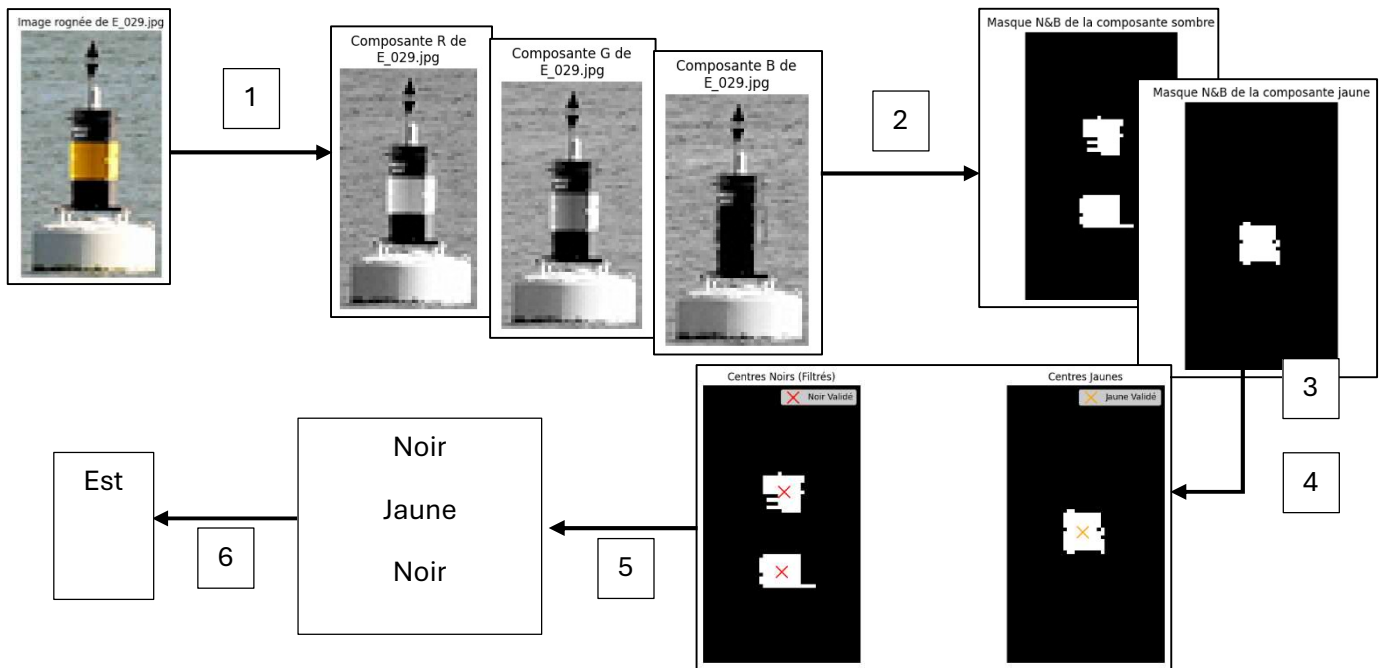
Si un cluster noir est trop proche d'un cluster jaune (on donne une tolérance de 5% dans notre programme), alors on le supprime.

5. Lecture de la séquence :

De haut en bas, on lit la séquence des clusters noir et jaune selon le post-traitement.

6. Décision :

La lecture de la séquence permet de connaître le cardinal correspondant (Nord : $N > J$, Sud : $J > N$, Est : $N > J > N$, Ouest : $J > N > J$).

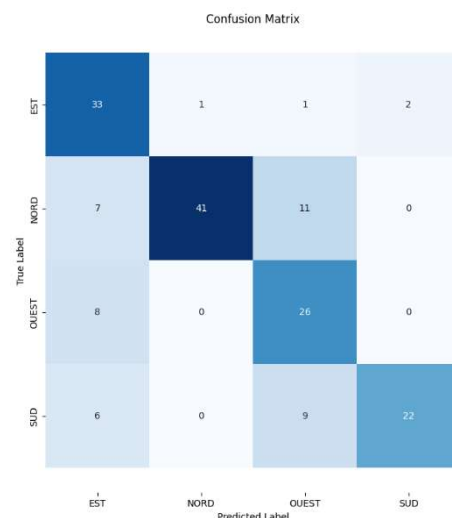


Résultats

On obtient pour les 167 images une accuracy globale de 73.05%.

On obtient la matrice de confusion ci-contre :

Les résultats sont globalement satisfaisants pour une maison "faite à la main" : 3/4 des bouées ont bien été classés.



Conclusion

L'atout majeur de cette méthode est sa grande souplesse et sa capacité de généralisation par rapport à l'Approche 1. Contrairement à des règles ("si le noir est en haut, c'est Nord") qui échouent au moindre imprévu, le modèle statistique est capable de tolérer des variations de forme, de légères inclinaisons de la bouée, ou des occultations partielles.

L'inconvénient principal de cette classification est sa forte dépendance à l'étape d'extraction des caractéristiques et à la qualité des recadrages. Si la boîte englobante générée par l'étape de détection est trop large, le modèle va intégrer le ciel et la mer dans ses calculs.

On aurait pu tester cette segmentation selon l'espace HSV qui est peut-être plus robuste pour faire le seuillage que celle RGB.

Approche 3 : Deep Learning orienté analyse d'image

Tâche de reconnaissance 1 : Détection de balise cardinale

Pour la détection de balise cardinale avec l'approche de DL orienté analyse d'image, on a utilisé la méthode de fine-tuning du modèle YOLOv8n vue dans le TD13 du module « Advanced Image Analysis » du bloc « Image Science (1) ».

Démarche

Notre démarche suit la même approche que celui du TD c'est-à-dire :

1. Organisation de l'arborescence :

On a créé un dataset respectant une arborescence bien spécifique à YOLO en séparant les données d'entraînement et de validation et en ajoutant un fichier .yaml qui permet à l'algorithme de savoir la vérité terrain (balise cardinale). On a choisi de séparer le dataset en deux ensembles : train et val avec pour pourcentage respectifs 80% et 20%. On a aussi fait en sorte qu'il y ait à peu près le même ratio de balises d'une même classe dans les deux ensembles. Pour le fichier .yaml, on a précisé une seule classe qui est 0 et qui correspond à la balise qu'on a nommé buoy.

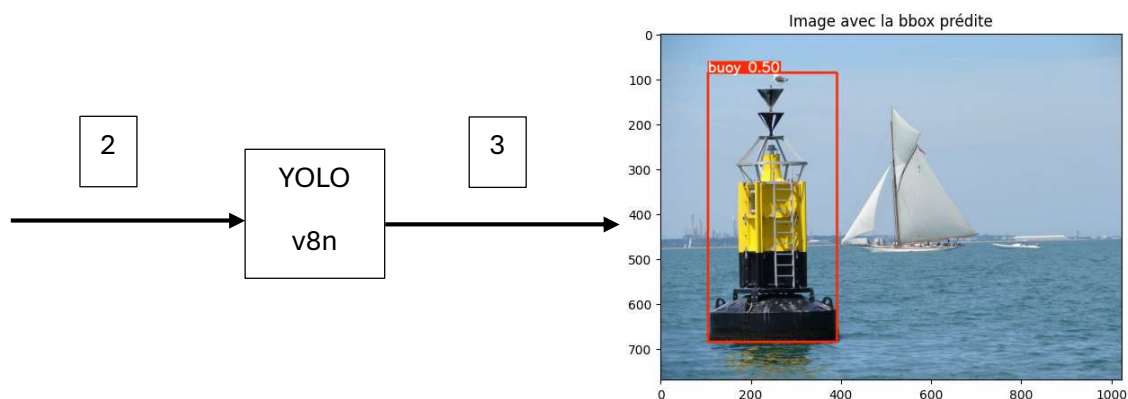
```
buoys0/  
├── images/  
│   ├── train/  
│   └── val/  
├── labels/  
│   ├── train/  
│   └── val/  
└── buoys0.yaml
```

2. Entraînement du réseau :

On entraîne le réseau pré-entraîné YOLOv8n avec nos images d'entraînement et des paramètres bien précis (learning rate = $1e-3$, optimizer = Adam, epochs = 3, batch = 16) pour détecter les boîtes.

3. Application du réseau :

On met en pratique le réseau pour la détection des balises cardinales des images de validation en mettant un niveau de confiance bas (on ne veut pas faire de classification pour l'instant donc on peut se le permettre).



Résultats

On obtient pour les 84 images test :

- Mean IoU: 0.7009
- mAP @ 0.5: 0.7647

Cette moyenne d'IoU est très satisfaisante pour la rapidité de l'algorithme.

Conclusion

Théoriquement, l'avantage d'un modèle comme YOLO est sa robustesse et sa vitesse d'exécution. En prédisant les boîtes englobantes de manière globale via une grille (et non plus en scannant laborieusement l'image avec une fenêtre glissante comme dans l'Approche 2), le modèle réduit les temps de calcul tout en comprenant le contexte global de l'image autour de la bouée.

L'inconvénient critique de cette méthode (scores de confiance entre 0.01 et 0.03) est sa sensibilité à la configuration technique et à la préparation des données. Une simple incompatibilité dans le format des annotations ou une absence d'initialisation des poids conduit à l'échec du réseau, rendant la détection inexploitable.

On aurait pu entraîner le réseau sur plus d'époques (au moins 10) et en implémentant d'autres paramètres pour qu'il soit plus robuste.

Tâche de reconnaissance 2 : Classification de direction cardinale

Pour la classification de direction cardinale avec l'approche de DL orienté analyse d'image, on a utilisé la méthode de fine-tuning du réseau de classification ResNet-18 vue dans le Lab10_XAI du module « Advanced Deep Learning » du bloc « Data Science (1) ».

Démarche

Notre démarche suit la même approche que celui du TD c'est-à-dire :

1. Préparation des données :

On organise l'arborescence pour l'entraînement. On crée les datasets en séparant les données d'entraînement, de validation et de test ainsi que les dataloaders respectifs. On choisit pour ratio d'images train/val/test respectivement 70%/15%/15%.

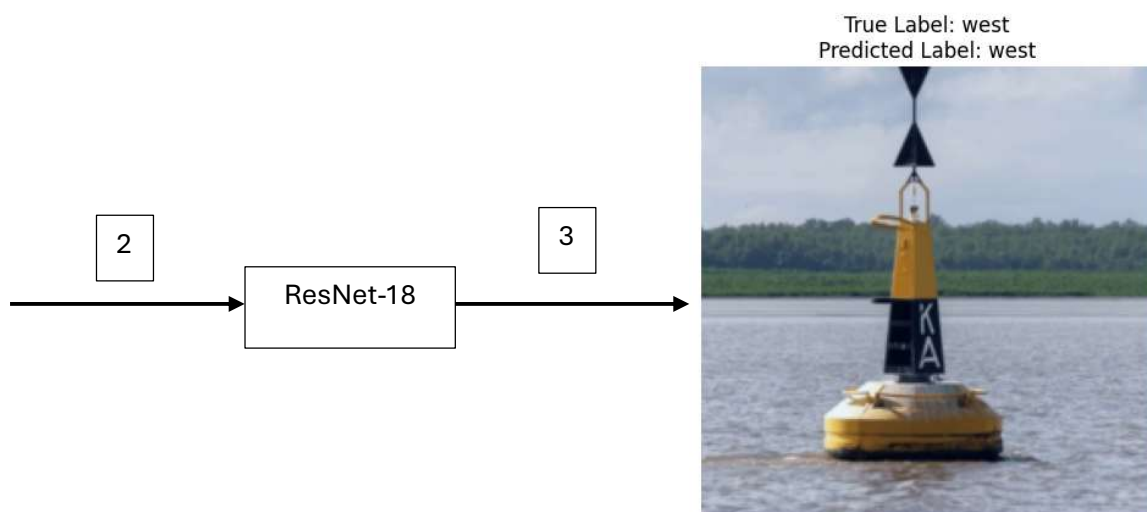
```
Project dataset - DL/  
├─ train/  
│   ├── class1/  
│   ├── class2/  
│   └── ...  
├─ val/  
│   ├── class1/  
│   ├── class2/  
│   └── ...  
└─ test/  
    ├── class1/  
    ├── class2/  
    └── ...
```

2. Fine-tuning :

On importe le modèle pré-entraîné ResNet-18 puis on gèle les premières couches afin de l'entraîner sur notre base d'images. On garde le gradient actif des paramètres sur les dernières couches (layer4 et fc). On choisit pour paramètres d'entraînement : learning rate = $1e-4$, optimizer = Adam, epochs = 20, batch-size = 8. On implémente aussi des transformations d'images telles que le redimensionnement en 256x256 ou la normalisation au niveau des canaux.

3. Application du réseau :

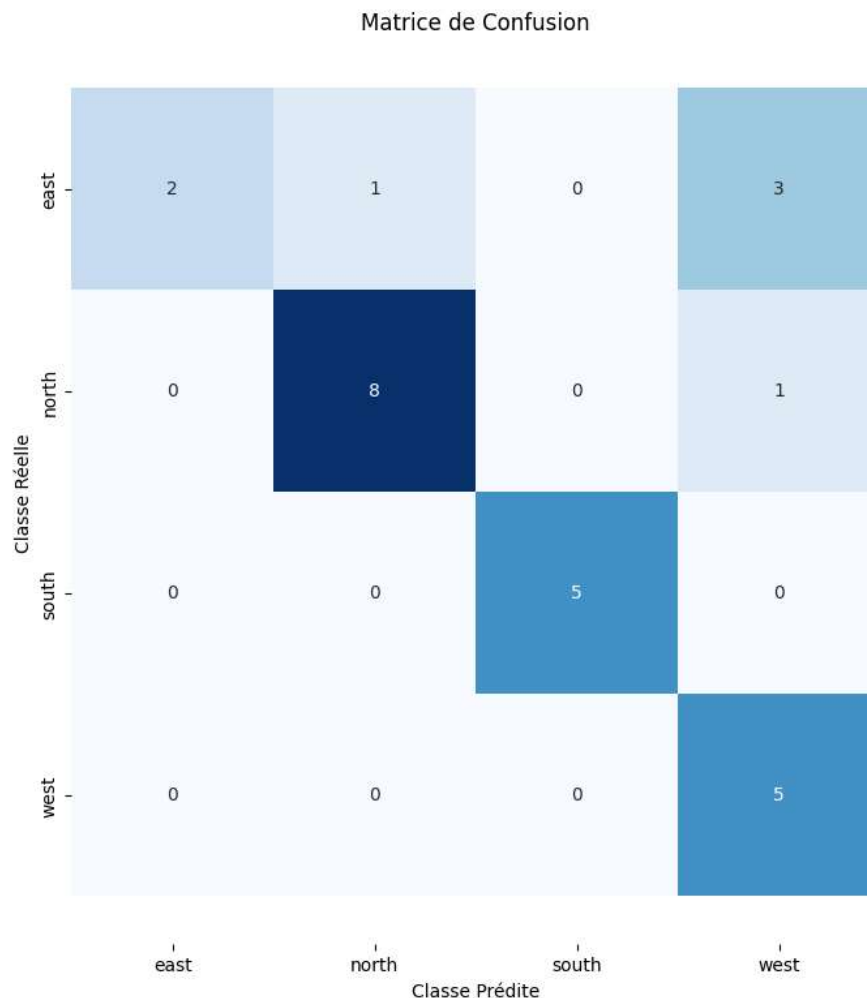
On met en pratique le réseau pour la classification des directions cardinales des images de test.



Résultats

On obtient pour les 25 images de test une accuracy globale de 80%.

On obtient la matrice de confusion ci-dessous :



Les résultats sont globalement satisfaisants : la précision globale surpasse les deux autres méthodes de classification.

Conclusion

L'avantage majeur est son ratio performance/temps d'apprentissage, rendu possible par la technique du fine tuning. En utilisant un modèle ResNet-18 pré-entraîné, il a suffi de réentraîner les dernières couches sur quelques époques pour obtenir des résultats.

L'inconvénient à cette puissance prédictive est la black-box des réseaux de neurones profonds. Par exemple, lors de notre soutenance, le réseau a mal prédit une classe et nous ne pouvons pas savoir d'où le problème vient, qu'est-ce que le réseau a regardé pour décider...

Comparaison des approches

Scores

Résultats globaux de chaque approche :

Détection	Mean IoU	mAP @0.50
Approche 1 (Image)	0.5991	0.6886
Approche 2 (ML)	0.5393	0.7024
Approche 3 (DL)	0.7009	0.7647

En classification, les grandeurs comme Precision ou Recall se calculent par classe, on les écrira alors dans cette ordre : Nord, Est, Sud, Ouest et en pourcentage (on peut se référer aux matrices de confusion disponibles au-dessus).

Classifi- cation	Precision	Recall	F1-Score	Accuracy	Accuracy globale
Approche 1	84, 59, 97, 93	78, 87, 78, 77	81, 70, 87, 84	87, 84, 95, 94	79.84%
Approche 2	98, 61, 92, 55	69, 89, 59, 76	81, 73, 72, 64	89, 85, 90, 83	73.05%
Approche 3	89, 100, 100, 56	89, 33, 100, 100	89, 50, 100, 71	92, 84, 100 , 84	80.00%

Interprétation

Si on devait classer les 3 approches selon les scores globaux, on mettrait dans l'ordre :

1. Approche Deep Learning (détection)
2. Approche Image Classique
3. Approche Machine Learning

Le classement des scores des extrémaux (ML et DL) peut s'expliquer de la façon suivante :

- Approche DL : Détection : Le modèle YOLO est principalement conçu pour répondre aux besoins de détection de boîtes englobantes, c'est pourquoi il donne le meilleur score.
- Approche DL : Classification : Le modèle entraîné de ResNet a donné les meilleurs résultats en classification.
- Approche ML : Détection : L'utilisation de l'opération de fusion de boîtes a tendance à trop prédire donc à avoir une boîte englobante. Il est aussi très sensible aux perturbations dû aux éléments alentours (bateau, île, ville).
- Approche ML : Classification : Il arrive que l'algorithme des K-Means interprète mal le centre des clusters et donc alterne l'ordre que la séquence était censée avoir.
- En général, les approches faits « à la main » sont sensibles aux bruits (comme pour la mer qui peut paraître sombre ou le ciel paraissant jaune). Ils sont aussi moins efficaces pour des balises très éloignées lorsqu'on ne connaît pas la vérité terrain.

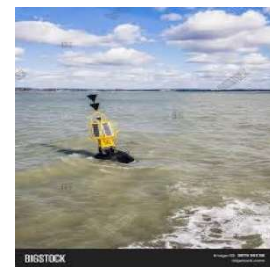
Coût computationnel

Nous pouvons lister ci-dessous quelques remarques spécifiques à certaines approches :

- Initialement, nous avons prévu de faire l'approche 3 entièrement avec le modèle YOLO (détection et classification). Le coût computationnel pour avoir les classes avec YOLO était déjà de l'ordre de quelques minutes et les prédictions étaient très mauvaises : le niveau de confiance était bas pour beaucoup de classes (de l'ordre de 0.01-0.03) et quand YOLO prédisait une classe, c'était rarement celui de la vérité terrain. Ceci est une des raisons pour laquelle nous avons choisi ResNet-18 pour la classification.
- Pour faire le lancement de la mesure de toutes les données de validation avec SVM (approche 2, détection), le programme a pris 5 minutes pour faire l'opération sur toutes les images de test. Cela fait de lui le moins optimisé pour traiter un large nombre de données.
- Si on prend en compte le temps d'exécution, la puissance computationnelle (utilisation du CPU/GPU), l'approche la plus rentable est la première car elle utilise des opérations simples et légères pour un ordinateur contrairement à l'entraînement de réseau comme ResNet-18, YOLO ou l'extraction de caractéristiques comme HOG.

Application des approches pour une image test

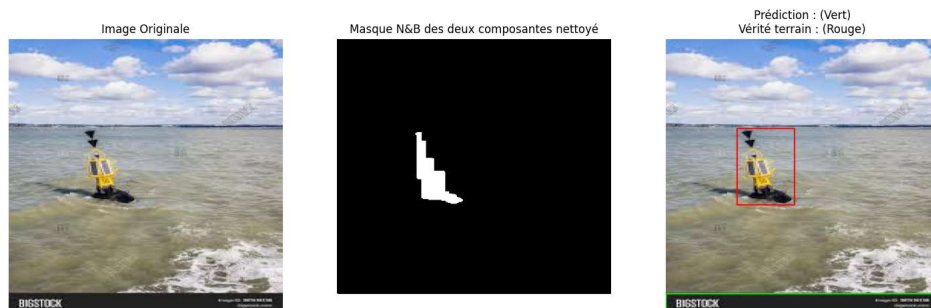
Pour la soutenance, notre image était la suivante : SOUTH_2.jpg
Avec pour fichier texte : 2 0.364844 0.467187 0.210938 0.278125



Nous allons vous présenter dans cette partie les résultats et nos interprétations pour chaque approche.

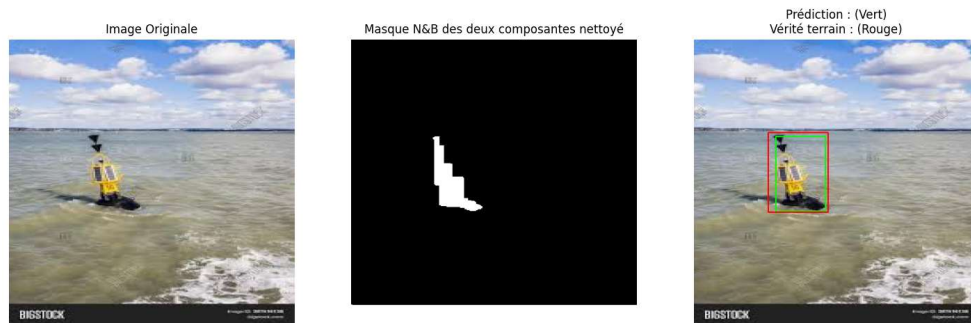
Approche 1 : Détection

Nos résultats sont les suivants :



Il est évident que la raison pour laquelle l'algorithme n'a pas détecté la balise est parce que la composante contenant du noir et du jaune ayant la plus grande aire est la banderole noire en bas de l'image.

Nous nous sommes rendu compte qu'il y a eu une erreur dans notre programme. En effet, nous avons mis originalement le ratio hauteur/largeur entre 2 et 8 et dans cette image, comme on remarque que l'image n'est pas 2 fois plus haute que large (un peu moins), alors elle ne prend pas en compte cette condition et choisit par défaut la composante ayant la plus grande aire. Or, pour faire les statistiques de détection, nous avons optimisé cette échelle de ratio en 1.5 à 8 pour avoir les meilleurs résultats globaux mais nous avons oublié de le mettre pour la partie « pas à pas » donc cela n'a pas pris en compte notre optimisation. Dans le cas où on a un ratio hauteur/largeur entre 1.5 et 8, nous avons ces résultats :



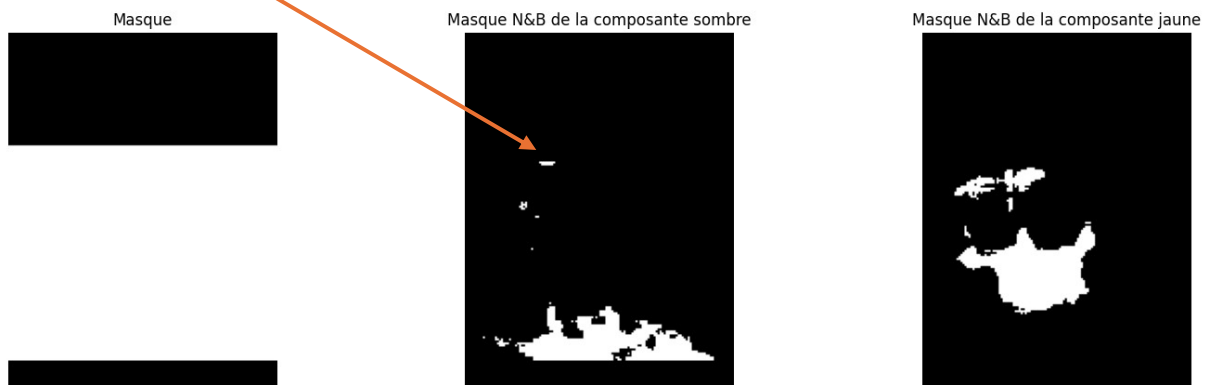
Le programme a alors bien trouvé la balise dans ce cas s'il n'y avait pas l'erreur contrairement à la soutenance. On trouve alors un IoU de 77.86% pour cette image ce qui est très satisfaisant.

Approche 1 : Classification

Voici ci-contre le résultat de la classification :

Le résultat n'est pas le bon à cause de ce morceau noir :

En effet, comme l'approche se base sur l'enchaînement couleur, après application du masque enlevant le cône, on a la couleur de plus haut niveau qui est la composante noire.



On a alors prédit l'enchaînement N>J>N (Est) plutôt que J>N (Sud)

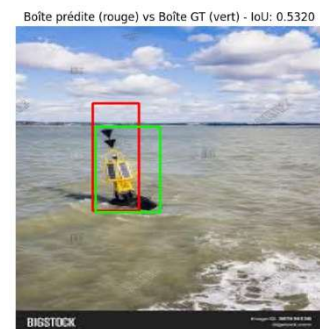
Approche 2 : Détection

Voici nos résultats pour cette approche :



Les résultats sont très satisfaisants car on voit bien que la balise est dans la boîte englobante prédite.

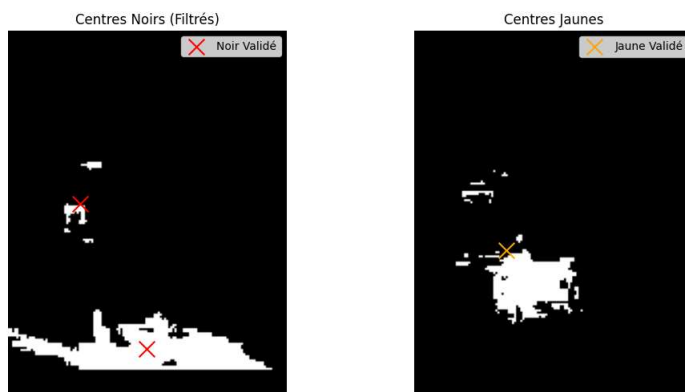
L'IoU de cette détection est de 53.20% ce qui est convenable pour cette approche.



Approche 2 : Classification

Voici le résultat de la classification ci-contre :

L'explication de ce résultat vient de la présence des panneaux solaires qui sont considérés comme une partie sombre de l'image et comme la partie jaune est concentrée entre les panneaux solaires et le bas de la balise, alors l'algorithme des K-Means a imposé un cluster noir sur le haut de l'image, ce qui ne donne l'enchaînement N>J>N (Est) plutôt que J>N (Sud).

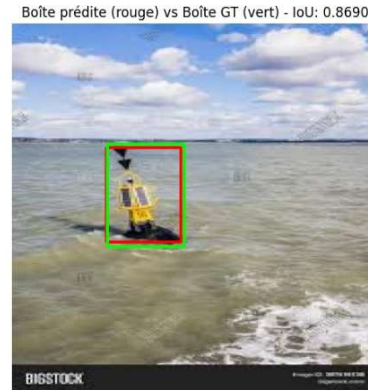


Approche 3 : Détection

Voici le résultat de la détection via YOLO :

Le résultat est très satisfaisant car on dirait que les deux boîtes englobantes se superposent. C'est le meilleur résultat de la soutenance.

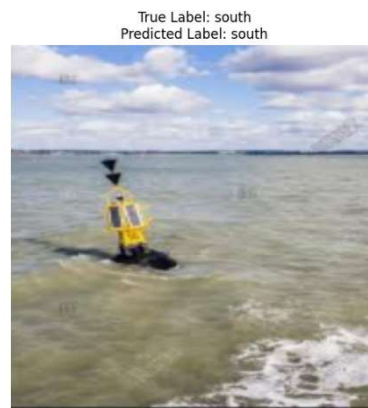
L'IoU est de 86.90%, ce qui est un très bon score.



Approche 3 : Classification

Voici le résultat avec la classification de ResNet-18 :

La prédiction est correcte pour cette balise. Nous ne pouvons pas interpréter la raison de cette bonne prédiction comme expliqué plus tôt dans le rapport (boîte noire des réseaux de Deep Learning).



Conclusion générale

Ce projet nous a permis d'explorer pas à pas différentes technologies de vision par ordinateur pour la détection et la classification de balises cardinales, mettant en évidence le compromis constant entre précision, robustesse et coût computationnel en ingénierie.

Si l'approche classique par traitement d'image s'est révélée très rapide mais fragile face aux variations environnementales (luminosité, échelle), l'approche par Machine Learning (HOG + SVM) a apporté une meilleure généralisation au prix d'un temps d'exécution plus long rendant son utilisation en temps réel impossible. Enfin, l'approche par Deep Learning a offert des résultats très satisfaisants : la classification du ResNet-18 et la détection avec YOLO ont donné des résultats très satisfaisants. Le principal axe d'amélioration consistera à implémenter massivement de la Data Augmentation afin de forcer nos modèles à devenir invariants aux conditions maritimes difficiles (houle, contre-jour, occultations). Nous devons aussi implémenter pour les approches telles que Deep Learning des seeds ou l'exportation des modèles pour garder les mêmes résultats.

En conclusion, ce projet a parfaitement rempli son rôle pédagogique. Il nous a confrontés aux réalités du prétraitement des données, à la nécessité de justifier ses choix architecturaux par des métriques rigoureuses, et à la gestion de l'effet de boîte noire inhérent à l'intelligence artificielle moderne.